

DATA STRUCTURES & ALGORITHMS LAB PROJECT

(1 Credit Hour)

CSL-221



DEPARTMENT OF INFORMATION TECHNOLOGY

BAHRIA UNIVERSITY KARACHI CAMPUS

Project Title:

**CITY WASTE MANAGEMENT
SYSTEM**

Submitted by:

Owais Khan Mughal	02-235232-042
Kanwer Farasat Ali	02-235232-012
M. Saim	02-235232-018

Submitted To:

Sr. Lecturer Faiza Khan

Submitted On:

2-Jan-2025

Acknowledgement

We would want to sincerely thank Ma'am Faiza Khan, who teaches us in the lab. The success of this City Waste Management System was largely due to her direction, encouragement, and insightful criticism.

Table of Contents

1) Abstract:	4
2) Introduction:	4
3) Problem statement:	4
4) Existing Systems:	4
5) Objectives and Scope of the Project:	5
6) Overview of project:	5
8) Tools and Technologies:	7
9) Project features:	7
10) Data Structures & Algorithms Used:	7
12) Output:	28
13) Conclusion:	41
14) References:	41

1) Abstract:

The Smart City Waste Management System aims to streamline garbage collection and resource optimization for urban centers. It employs advanced data structures. The Smart City Waste Management System is designed to automate and optimize urban waste collection and resource management. Using efficient algorithms and data structures, the system manages city areas, trucks, and complaints to ensure seamless operations. Features such as dynamic truck routing, real-time complaint handling, and detailed reporting make the system highly efficient and user-friendly. The integration of AVL trees, min-heaps, and graph algorithms allows the system to process data quickly and adapt to changing urban scenarios. This project aims to improve city cleanliness, reduce operational costs, and enhance citizen satisfaction by addressing complaints in a prioritized manner.

2) Introduction:

The Smart City Waste Management System is an innovative approach to tackling the growing waste management issues in urban centers. With cities expanding and populations increasing, efficient waste management is critical for maintaining public health and environmental sustainability. This system leverages modern algorithms and data structures to ensure efficient and timely waste collection while addressing citizen complaints effectively.

3) Problem statement:

Urban areas often suffer from inefficient waste collection systems. Common issues include poorly planned truck routes, delayed complaint handling, and inadequate resource allocation. These inefficiencies lead to increased operational costs, citizen dissatisfaction, and adverse environmental effects. A streamlined and automated solution is needed to address these challenges.

4) Existing Systems:

Current waste management solutions either rely on manual processes or use outdated systems with limited functionality. These systems often fail to optimize truck routes or handle citizen complaints effectively. Advanced systems are available but are either too expensive or lack modularity, making them unsuitable for widespread adoption.

5) Objectives and Scope of the Project:

- **Objective:**

The primary objective of this project is to create an efficient, scalable, and cost-effective waste management system. The system is designed to:

- Automate the management of city areas and garbage collection.
- Optimize truck routing to reduce fuel consumption and time.
- Prioritize and address citizen complaints effectively.
- Generate comprehensive reports for better decision-making.

- **Scope**

The system covers multiple aspects of city waste management:

- **Area Management:** Tracks population, waste generation, and growth rates.
- **Truck Management:** Allocates resources dynamically and plans routes.
- **Complaint Handling:** Provides a priority-based system for addressing issues.
- **Reporting:** Generates analytics and insights to improve future operations.

By automating these processes, the system ensures a cleaner and healthier urban environment.

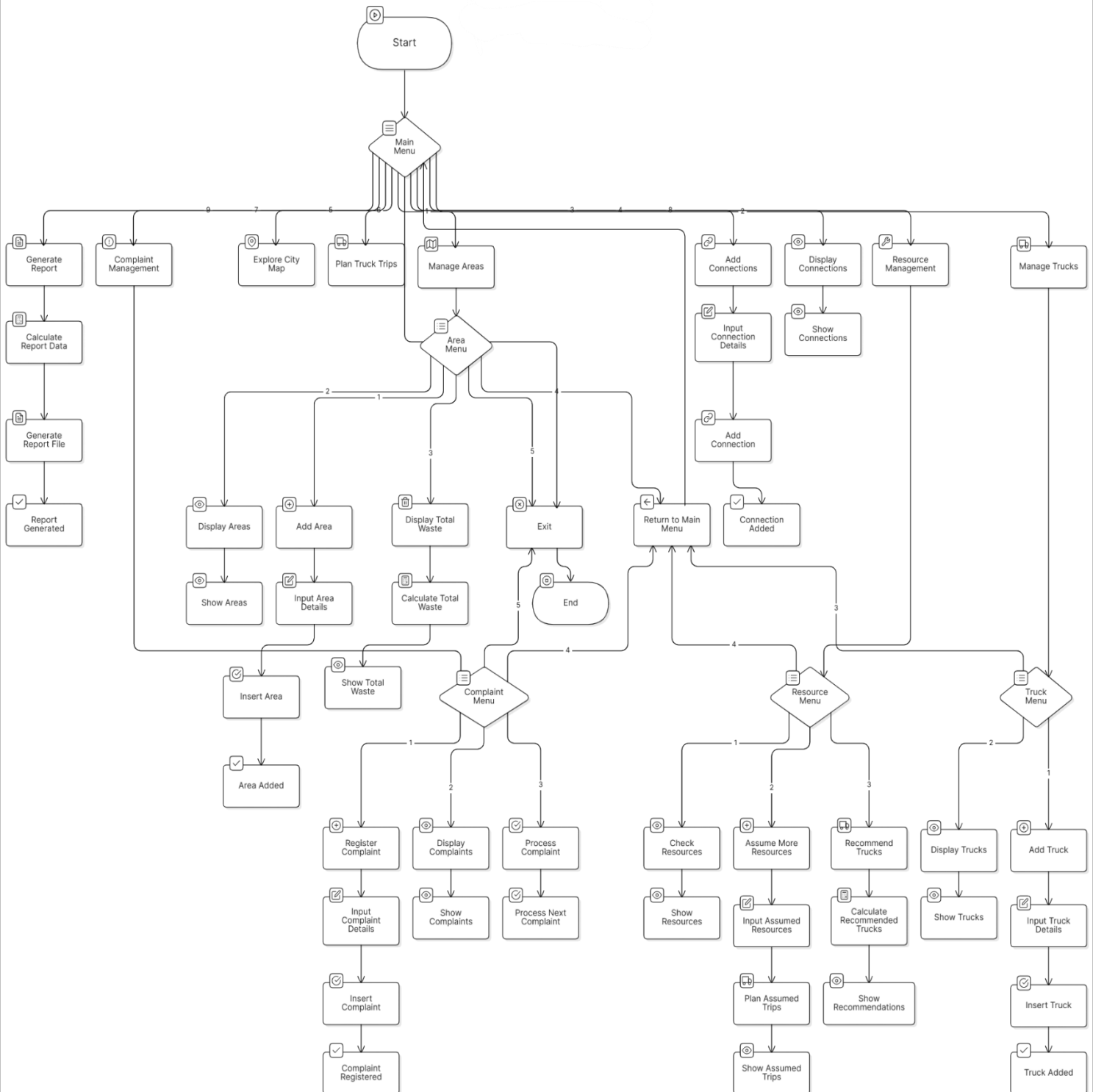
6) Overview of project:

Core features within the following scope are the system's main focus:

The Smart City Waste Management System is divided into three main modules: area management, truck scheduling, and complaint handling. The area management module tracks waste generation using population data and growth rates. The truck scheduling module assigns trucks dynamically based on garbage volume and optimizes routes using graph-based algorithms. The complaint handling module uses a priority queue to address issues based on urgency.

The system also includes a reporting feature that provides detailed analytics, including trip statistics, fuel costs, and garbage collection trends. These insights help administrators plan better and make informed decisions. Designed with modularity in mind, the system can easily adapt to the specific needs of different cities.

7) Workflow:



8) Tools and Technologies:

- **Language:** C++ was selected for its speed and efficiency in implementing complex algorithms and data structures.
- **Integrated Development Environment:** Microsoft Visual Studio was used for coding, debugging, and compiling the project. Its robust tools and user-friendly interface made it ideal for this development process.
- **Libraries and Utilities:** Standard C++ libraries for input/output, file handling, and algorithms were utilized for core functionalities.

9) Project features:

- **Dynamic Area Management:** Adds and displays city areas, tracks population, and calculates garbage generation based on growth rates.
- **Truck Scheduling:** Dynamically allocates trucks, plans routes, and calculates distances using graph-based algorithms.
- **Route Optimization:** Employs Dijkstra's algorithm to find the shortest path between areas, minimizing fuel costs and time.
- **Complaint Handling:** Implements a min-heap to prioritize citizen complaints by urgency, ensuring timely resolution.
- **Analytics and Reporting:** Generates detailed reports, including total garbage collected, truck trips, distances traveled, and fuel costs.
- **Resource Recommendations:** Suggests the number of trucks required based on garbage data and truck capacity assumptions.

10) Data Structures & Algorithms Used:

1. AVL Tree for Area Management:

- **Description:** A self-balancing binary search tree that ensures efficient insertion and retrieval of area data.
- **Reason for Use:** Ensures $O(\log n)$ complexity for operations, making it suitable for dynamic datasets with frequent updates.
- **Complexity:**
 $O(\log n)$

2. Min-Heap for Complaint Handling:

- **Description:** A binary heap that prioritizes complaints based on urgency.
- **Reason for Use:** Guarantees that the highest-priority complaint is addressed first.
- **Complexity:**
 - $O(\log n)$

3. Graph Algorithms for Route Optimization:

- **Description:** *Dijkstra's* algorithm calculates the shortest path between areas.
- **Reason for Use:** Optimizes truck routes, reducing operational costs and time.
- **Complexity:**
 - Time: $O(V^2)$ with an adjacency matrix, suitable for the given problem size.

4. Round Robin:

- Implemented implicitly via arrays or queues for cyclic allocation of trucks, ensuring all trucks are utilized fairly.
- Time complexity of $O(1)$ per truck.

Data Structure	Purpose	Insertion	Deletion	Search	Additional Notes
AVL Tree	Area management (population, garbage tracking)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Ensures balanced tree for efficient operations.
Min-Heap	Complaint management (prioritized by urgency)	$O(\log n)$	$O(\log n)$	N/A	Used for extracting complaints with highest priority.
Graph (Adjacency List)	Area connections and route optimization	$O(1)$ per edge	$O(E)$ for all edges	$O(V+E)$ for traversal	Efficient representation for Dijkstra's algorithm.
Graph (Dijkstra's Algorithm)	Route optimization (shortest path)	N/A	N/A	$O(V^2)$	Used for truck route optimization.
Round Robin (Array/Queue)	Allocating trucks for garbage collection in cycles	$O(1)$	$O(1)$	$O(1)$	Ensures fairness and cyclic truck allocation.

Assumptions Made:

Truck Capacity: A standard capacity of 2000 kg was assumed for truck recommendations. This ensures consistency in planning and aligns with industry norms.

Waste Growth Rates: Default rates were set based on generic urban data for simplicity and scalability.

Static Graph Model: The city layout was considered static, with pre-defined connections and distances between areas, but also user can do it.

Fuel Costs: A fixed rate of Rs. 190 per kilometer was assumed for calculating operational expenses.

These assumptions simplify the initial implementation while ensuring the system remains adaptable to real-world scenarios.

11) Code:

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <string>
#include <vector>
#include <limits>
#include <algorithm>
#include <fstream>
using namespace std;

struct AreaNode {
    string areaName;
    int population;
    double wasteGrowthRate;
    double garbagePerDay;
    AreaNode* left;
    AreaNode* right;
    int height;
};

int height(AreaNode* node) {
    return node ? node->height : 0;
}

int getBalanceFactor(AreaNode* node) {
    return node ? height(node->left) - height(node->right) : 0;
}

AreaNode* createNode(string name, int population, double wasteGrowthRate) {
    AreaNode* node = new AreaNode();
    node->areaName = name;
    node->population = population;
    node->wasteGrowthRate = wasteGrowthRate;
    node->garbagePerDay = population * wasteGrowthRate;
    node->left = node->right = nullptr;
    node->height = 1;
    return node;
}
```

```

AreaNode* rightRotate(AreaNode* y) {
    AreaNode* x = y->left;
    AreaNode* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

AreaNode* leftRotate(AreaNode* x) {
    AreaNode* y = x->right;
    AreaNode* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

AreaNode* insertArea(AreaNode* node, string name, int population, double wasteGrowthRate) {
    if (!node)
        return createNode(name, population, wasteGrowthRate);

    if (name < node->areaName)
        node->left = insertArea(node->left, name, population, wasteGrowthRate);
    else if (name > node->areaName)
        node->right = insertArea(node->right, name, population, wasteGrowthRate);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));

    int balance = getBalanceFactor(node);

    if (balance > 1 && name < node->left->areaName)
        return rightRotate(node);

    if (balance < -1 && name > node->right->areaName)
        return leftRotate(node);

    if (balance > 1 && name > node->left->areaName) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    if (balance < -1 && name < node->right->areaName) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

AreaNode* root = nullptr;

void displayAreas(AreaNode* root, double& totalGarbage) {
    if (!root)
        return;

    displayAreas(root->left, totalGarbage);
    cout << left << setw(20) << root->areaName

```

```

        << setw(15) << root->population
        << setw(20) << fixed << setprecision(2) << root->wasteGrowthRate
        << setw(20) << root->garbagePerDay << " kg" << endl;
totalGarbage += root->garbagePerDay;
displayAreas(root->right, totalGarbage);
}

void mainMenu();
double totalGarbage = 0;
void Areamenu() {

int choice;

do {
    cout << "\n\t\t Area Mangement Potal";
    cout << "\n\t\t===== \n\n";
    cout << "1. Add Area" << endl;
    cout << "2. Display Areas" << endl;
    cout << "3. Display Total City Waste" << endl;
    cout << "4. Main menu" << endl;
    cout << "5. Exit" << endl;
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
    case 1:
    {
        string name;
        int population;
        double wasteGrowthRate;
        cout << "Enter area name: ";
        cin >> name;
        cout << "Enter population: ";
        cin >> population;
        cout << "Enter waste growth rate (kg per person per day): ";
        cin >> wasteGrowthRate;
        root = insertArea(root, name, population, wasteGrowthRate);
        cout << "Area added successfully!\n";
        break;
    }
    case 2:
    {
        totalGarbage = 0;
        cout << "\n\n";
        cout << left << setw(20) << "Area Name"
            << setw(15) << "Population"
            << setw(20) << "Waste Growth Rate"
            << "Garbage per Day" << endl;
        cout << string(75, '-') << endl;
        displayAreas(root, totalGarbage);
        break;
    }
    case 3:
    {
        totalGarbage = 0;
        cout << "\n\n";
        cout << left << setw(20) << "Area Name"
            << setw(15) << "Population"
            << setw(20) << "Waste Growth Rate"
            << "Garbage per Day" << endl;
        cout << string(75, '-') << endl;
        displayAreas(root, totalGarbage);
        cout << "\nTotal City Garbage per Day: " << fixed << setprecision(2) << totalGarbage << " kg (Tons: " << totalGarbage /
1000 << " )\n\n";
        break;
    }
    case 4:
    {

```

```

        cout << "Returning to main menu...\n";
        mainMenu();
        break;
    }
    case 5:
        cout << "Exiting...\n";
        exit(0);
        break;
    default:
        cout << "Invalid choice. Please try again.\n";
    }
} while (choice != 4);
}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// TRUCK MODULE

struct TruckNode {
    string truckID;
    int capacity;
    TruckNode* left;
    TruckNode* right;
    int height;
};

int truckHeight(TruckNode* node) {
    return node ? node->height : 0;
}

int getTruckBalanceFactor(TruckNode* node) {
    return node ? truckHeight(node->left) - truckHeight(node->right) : 0;
}

TruckNode* createTruckNode(string id, int capacity) {
    TruckNode* node = new TruckNode();
    node->truckID = id;
    node->capacity = capacity;
    node->left = node->right = nullptr;
    node->height = 1;
    return node;
}

TruckNode* rightRotateTruck(TruckNode* y) {
    TruckNode* x = y->left;
    TruckNode* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(truckHeight(y->left), truckHeight(y->right)) + 1;
    x->height = max(truckHeight(x->left), truckHeight(x->right)) + 1;

    return x;
}

TruckNode* leftRotateTruck(TruckNode* x) {
    TruckNode* y = x->right;
    TruckNode* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(truckHeight(x->left), truckHeight(x->right)) + 1;
    y->height = max(truckHeight(y->left), truckHeight(y->right)) + 1;

    return y;
}

```

```

TruckNode* insertTruck(TruckNode* node, string id, int capacity) {
if (!node) {
    cout << "\n--- Inserting truck: " << id << " with capacity " << capacity << " kg. ---\n";
    return createTruckNode(id, capacity);
}

if (id < node->truckID)
    node->left = insertTruck(node->left, id, capacity);
else if (id > node->truckID)
    node->right = insertTruck(node->right, id, capacity);
else {
    cout << "Duplicate truck ID not allowed: " << id << endl;
    return node;
}

node->height = 1 + max(truckHeight(node->left), truckHeight(node->right));

int balance = getTruckBalanceFactor(node);

if (balance > 1 && id < node->left->truckID)
    return rightRotateTruck(node);

if (balance < -1 && id > node->right->truckID)
    return leftRotateTruck(node);

if (balance > 1 && id > node->left->truckID) {
    node->left = leftRotateTruck(node->left);
    return rightRotateTruck(node);
}

if (balance < -1 && id < node->right->truckID) {
    node->right = rightRotateTruck(node->right);
    return leftRotateTruck(node);
}

return node;
}

TruckNode* truckRoot = nullptr;

void displayTrucks(TruckNode* root) {
if (!root)
    return;

displayTrucks(root->left);
cout << left << setw(15) << root->truckID
    << setw(10) << root->capacity << " kg" << endl;
displayTrucks(root->right);
}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// graph module

const int MAX_AREAS = 100;
const int INF = numeric_limits<int>::max();

struct Edge {
int destination;
int distance;
Edge* next;
};

struct Area {
string name;
int garbage;
Edge* head;
};

```

```

struct Graph {
    Area areas[MAX_AREAS];
    int areaCount;
};

void initializeGraph(Graph& graph) {
    graph.areaCount = 0;
    for (int i = 0; i < MAX_AREAS; ++i) {
        graph.areas[i].name = "";
        graph.areas[i].garbage = 0;
        graph.areas[i].head = nullptr;
    }

    graph.areas[graph.areaCount].name = "Dumping Station";
    graph.areas[graph.areaCount].garbage = 0;
    graph.areas[graph.areaCount].head = nullptr;
    graph.areaCount++;
}

int addArea(Graph& graph, string areaName, int garbage) {
    if (graph.areaCount >= MAX_AREAS) {
        cout << "Cannot add more areas. Maximum limit reached.\n";
        return -1;
    }
    graph.areas[graph.areaCount].name = areaName;
    graph.areas[graph.areaCount].garbage = garbage;
    graph.areas[graph.areaCount].head = nullptr;
    return graph.areaCount++;
}

int findAreaIndex(Graph& graph, string areaName) {
    for (int i = 0; i < graph.areaCount; ++i) {
        if (graph.areas[i].name == areaName)
            return i;
    }
    return -1;
}

void addConnection(Graph& graph, string sourceName, string destName, int distance) {
    int srcIndex = findAreaIndex(graph, sourceName);
    int destIndex = findAreaIndex(graph, destName);

    if (srcIndex == -1 || destIndex == -1) {
        cout << "Error: One or both areas not found.\n";
        return;
    }

    Edge* newEdge = new Edge{ destIndex, distance, graph.areas[srcIndex].head };
    graph.areas[srcIndex].head = newEdge;

    newEdge = new Edge{ srcIndex, distance, graph.areas[destIndex].head };
    graph.areas[destIndex].head = newEdge;

    cout << "Connection added between " << sourceName << " and " << destName << " with distance " << distance << " km.\n";
}

void displayGraph(Graph& graph) {
    cout << "\nAreas and their connections:\n";
    for (int i = 0; i < graph.areaCount; ++i) {
        cout << graph.areas[i].name << " (Garbage: " << graph.areas[i].garbage << " kg) -> ";
        Edge* current = graph.areas[i].head;
        while (current) {
            cout << graph.areas[current->destination].name << "(" << current->distance << " km) ";
            current = current->next;
        }
        cout << endl;
    }
}

```

```

vector<int> dijkstra(Graph& graph, int start, int target) {
    vector<int> dist(graph.areaCount, INF);
    vector<int> parent(graph.areaCount, -1);
    vector<bool> visited(graph.areaCount, false);

    dist[start] = 0;

    for (int i = 0; i < graph.areaCount - 1; ++i) {
        int u = -1;

        for (int j = 0; j < graph.areaCount; ++j) {
            if (!visited[j] && (u == -1 || dist[j] < dist[u])) {
                u = j;
            }
        }

        if (dist[u] == INF)
            break;

        visited[u] = true;

        for (Edge* edge = graph.areas[u].head; edge != nullptr; edge = edge->next) {
            int v = edge->destination;
            int weight = edge->distance;

            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                parent[v] = u;
            }
        }
    }

    vector<int> path;
    for (int at = target; at != -1; at = parent[at]) {
        path.push_back(at);
    }
    reverse(path.begin(), path.end());

    if (path.size() == 1 && path[0] != target) {
        return {};
    }

    return path;
}

string getTruckID(TruckNode* root, int index, int& count) {
    if (!root)
        return "";

    string left = getTruckID(root->left, index, count);
    if (!left.empty())
        return left;

    if (count == index)
        return root->truckID;
    count++;

    return getTruckID(root->right, index, count);
}

int getTruckCapacity(TruckNode* root, int index, int& count) {
    if (!root)
        return 0;

    int left = getTruckCapacity(root->left, index, count);
    if (left != 0)
        return left;

```

```

if (count == index)
    return root->capacity;
count++;

return getTruckCapacity(root->right, index, count);
}
string getTruckID(TruckNode* root, int index) {
int count = 0;
return getTruckID(root, index, count);
}

int getTruckCapacity(TruckNode* root, int index) {
int count = 0;
return getTruckCapacity(root, index, count);
}

int getTruckCount(TruckNode* root) {
if (!root)
    return 0;
return 1 + getTruckCount(root->left) + getTruckCount(root->right);
}
void displayTargetedAreas(Graph& graph) {
cout << "\nTargeted Areas for Collecting Trash:\n";
cout << left << setw(20) << "Area Name" << setw(20) << "Garbage (kg)" << endl;
cout << string(40, '-') << endl;
for (int i = 1; i < graph.areaCount; ++i) {
    if (graph.areas[i].garbage > 0) {
        cout << left << setw(20) << graph.areas[i].name << setw(20) << graph.areas[i].garbage << endl;
    }
}
}

void addAreasFromAVL(Graph& graph, AreaNode* root) {
if (!root)
    return;

addAreasFromAVL(graph, root->left);
if (findAreaIndex(graph, root->areaName) == -1) {
    addArea(graph, root->areaName, root->garbagePerDay);
}
addAreasFromAVL(graph, root->right);
}

int totalDistanceAllTrucks = 0;
int totalTrips = 0;

void planTruckTrips(Graph& graph) {
addAreasFromAVL(graph, root);

vector<int> originalGarbage(graph.areaCount);
for (int i = 0; i < graph.areaCount; ++i) {
    originalGarbage[i] = graph.areas[i].garbage;
}

cout << "\nPlanning truck trips:\n";
ofstream analyticsFile("analytics.txt", ios::out | ios::trunc);
if (!analyticsFile) {
    cerr << "Error opening analytics file.\n";
    return;
}

int totalAreas = graph.areaCount - 1;
int truckCount = getTruckCount(truckRoot);
totalTrips = 0;
vector<int> truckTrips(truckCount, 0);
vector<int> truckDistances(truckCount, 0);
totalDistanceAllTrucks = 0;

```



```

bool garbageLeft = true;

analyticsFile << "Areas Information:\n";
analyticsFile << left << setw(20) << "Area Name" << setw(15) << "Population" << setw(20) << "Waste Growth Rate" << "Garbage per Day
(kg)\n";
analyticsFile << string(75, '-') << endl;
for (int i = 1; i < graph.areaCount; ++i) {
    analyticsFile << left << setw(20) << graph.areas[i].name
        << setw(15) << graph.areas[i].garbage / root->wasteGrowthRate
        << setw(20) << root->wasteGrowthRate
        << setw(20) << graph.areas[i].garbage << " kg\n";
}
analyticsFile << endl;

analyticsFile << "Available Trucks:\n";
analyticsFile << left << setw(15) << "Truck ID" << "Capacity (kg)\n";
analyticsFile << string(25, '-') << endl;
for (int truckIndex = 0; truckIndex < truckCount; ++truckIndex) {
    string truckID = getTruckID(truckRoot, truckIndex);
    int truckCapacity = getTruckCapacity(truckRoot, truckIndex);
    analyticsFile << left << setw(15) << truckID << truckCapacity << " kg" << endl;
}
analyticsFile << endl;

analyticsFile << "Truck Trips:\n";
analyticsFile << string(75, '-') << endl;

while (garbageLeft) {
    garbageLeft = false;
    bool allGarbageCollected = true;
    int totalGarbageCollected = 0;

    displayTargetedAreas(graph);
    cout << "\n\n";
    cout << "\tAvailable Trucks" << endl;
    cout << "-----\n\n";
    displayTrucks(truckRoot);

    for (int truckIndex = 0; truckIndex < truckCount; ++truckIndex) {
        string truckID = getTruckID(truckRoot, truckIndex);
        int truckCapacity = getTruckCapacity(truckRoot, truckIndex);
        int remainingCapacity = truckCapacity;

        vector<pair<string, int>> route;
        route.push_back({ "Dumping Station", 0 });

        int currentAreaIndex = 0;
        int totalDistance = 0;

        while (remainingCapacity > 0) {
            int nextAreaIndex = -1;
            int minDistance = INF;
            int maxGarbage = 0;

            for (int areaIndex = 1; areaIndex < graph.areaCount; ++areaIndex) {
                if (graph.areas[areaIndex].garbage > 0) {
                    vector<int> path = dijkstra(graph, currentAreaIndex, areaIndex);
                    int distance = path.size() - 1;
                    if (distance < minDistance || (distance == minDistance &&
graph.areas[areaIndex].garbage > maxGarbage)) {
                        minDistance = distance;
                        nextAreaIndex = areaIndex;
                        maxGarbage = graph.areas[areaIndex].garbage;
                    }
                }
            }
        }
    }
}

```

```

        if (nextAreaIndex == -1) {
            break;
        }

        int garbage = graph.areas[nextAreaIndex].garbage;
        int load = min(garbage, remainingCapacity);
        remainingCapacity -= load;
        graph.areas[nextAreaIndex].garbage -= load;
        totalGarbageCollected += load;

        vector<int> pathToNextArea = dijkstra(graph, currentAreaIndex, nextAreaIndex);
        for (int areaIndex : pathToNextArea) {
            if (areaIndex != currentAreaIndex) {
                route.push_back({ graph.areas[areaIndex].name, 0 });
                totalDistance += graph.areas[currentAreaIndex].head->distance;
                currentAreaIndex = areaIndex;
            }
        }

        route.back().second = load;

        if (graph.areas[nextAreaIndex].garbage > 0) {
            garbageLeft = true;
        }
    }

    vector<int> pathToDumpingStation = dijkstra(graph, currentAreaIndex, 0);
    for (int areaIndex : pathToDumpingStation) {
        if (areaIndex != currentAreaIndex) {
            route.push_back({ graph.areas[areaIndex].name, 0 });
            totalDistance += graph.areas[currentAreaIndex].head->distance;
            currentAreaIndex = areaIndex;
        }
    }

    if (route.size() > 1 && route.back().first == "Dumping Station") {
        route.pop_back();
    }

    if (route.size() > 1) {
        truckTrips[truckIndex]++;
        totalTrips++;
        truckDistances[truckIndex] += totalDistance;
        totalDistanceAllTrucks += totalDistance;
        cout << "\nTruck ID: " << truckID << "\nRoute: \n";
        analyticsFile << "\nTruck ID: " << truckID << "\nRoute: \n";
        for (const auto& [area, collected] : route) {
            cout << area << " (" << collected << " kg) -> ";
            analyticsFile << area << " (" << collected << " kg) -> ";
        }
        cout << "Dumping Station (0 kg)\n\n";
        analyticsFile << "Dumping Station (0 kg)\n\n";
        cout << "Total Distance Traveled: " << totalDistance << " km\n";
        analyticsFile << "Total Distance Traveled: " << totalDistance << " km\n";
        cout << "Truck Capacity: " << truckCapacity << " kg, Returning with: " << remainingCapacity << " kg\n";
        analyticsFile << "Truck Capacity: " << truckCapacity << " kg, Returning with: " << remainingCapacity << " kg\n";
    }

    Capacity Left\n\n";
    Capacity Left\n\n";
}

for (int areaIndex = 1; areaIndex < graph.areaCount; ++areaIndex) {
    if (graph.areas[areaIndex].garbage > 0) {
        allGarbageCollected = false;
        break;
    }
}

if (allGarbageCollected || totalGarbageCollected == 0) {

```

```

        break;
    }
}

cout << "All areas serviced and garbage delivered to the dumping station.\n\n";
analyticsFile << "All areas serviced and garbage delivered to the dumping station.\n\n";

for (int truckIndex = 0; truckIndex < truckCount; ++truckIndex) {
    string truckID = getTruckID(truckRoot, truckIndex);
    cout << "Truck ID: " << truckID << " made " << truckTrips[truckIndex] << " trips and traveled " << truckDistances[truckIndex] << "
km.\n";
    analyticsFile << "Truck ID: " << truckID << " made " << truckTrips[truckIndex] << " trips and traveled " <<
truckDistances[truckIndex] << " km.\n";
}

cout << "\nTotal number of trips by all trucks: " << totalTrips << "\n";
cout << "Total distance traveled by all trucks: " << totalDistanceAllTrucks << " km\n";
analyticsFile << "\nTotal number of trips by all trucks: " << totalTrips << "\n";
analyticsFile << "Total distance traveled by all trucks: " << totalDistanceAllTrucks << " km\n";

analyticsFile.close();

for (int i = 0; i < graph.areaCount; ++i) {
    graph.areas[i].garbage = originalGarbage[i];
}
}
Graph graph;

////////////////////////////////////
// COMPLAINT MODULE

struct Complaint {
    int id;
    string category;
    string description;
    int priority;
};

struct MinHeap {
    Complaint complaints[100];
    int size;

    MinHeap() : size(0) {}

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (complaints[parent].priority > complaints[index].priority) {
                swap(complaints[parent], complaints[index]);
                index = parent;
            }
            else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int smallest = index;
        int left = 2 * index + 1;
        int right = 2 * index + 2;

        if (left < size && complaints[left].priority < complaints[smallest].priority)
            smallest = left;

        if (right < size && complaints[right].priority < complaints[smallest].priority)
            smallest = right;
    }
}

```

```

        if (smallest != index) {
            swap(complaints[index], complaints[smallest]);
            heapifyDown(smallest);
        }
    }

void insert(const Complaint& complaint) {
    if (size >= 100) {
        cout << "Heap is full. Cannot register more complaints.\n";
        return;
    }
    complaints[size] = complaint;
    heapifyUp(size);
    size++;
}

Complaint extractMin() {
    if (size == 0) {
        cout << "No complaints to display.\n";
        return { -1, "", "", -1 };
    }
    Complaint minComplaint = complaints[0];
    complaints[0] = complaints[size - 1];
    size--;
    heapifyDown(0);
    return minComplaint;
}

bool isEmpty() const {
    return size == 0;
}

};

MinHeap complaintHeap;

int getValidatedUrgency() {
    int urgency;
    do {
        cout << "Enter urgency level (1-5): ";
        cin >> urgency;
        if (urgency < 1 || urgency > 5) {
            cout << "Invalid input. Please enter a value between 1 and 5.\n";
        }
    } while (urgency < 1 || urgency > 5);
    return urgency;
}

void saveToFile(const Complaint& complaint) {
    ofstream outFile("complaints.txt", ios::app);
    if (outFile) {
        outFile << "ID: " << complaint.id << " | Category: " << complaint.category
            << " | Description: " << complaint.description << " | Priority: " << complaint.priority << endl;
        outFile.close();
    }
    else {
        cout << "Error saving to file.\n";
    }
}

void registerComplaint(int& idCounter) {
    Complaint c;
    c.id = idCounter++;
    c.category = getCategory();
    cout << "Enter description: ";
    cin.ignore();
    getline(cin, c.description);
    c.priority = getValidatedUrgency();
}

```

```

complaintHeap.insert(c);
saveToFile(c);

cout << "Complaint registered successfully!\n";
}

void displayComplaints() {
if (complaintHeap.isEmpty()) {
    cout << "No complaints registered yet.\n";
    return;
}

cout << "\n--- Complaints (Prioritized by Urgency) ---\n";
MinHeap tempHeap = complaintHeap;
while (!tempHeap.isEmpty()) {
    Complaint top = tempHeap.extractMin();
    cout << "ID: " << top.id
        << " | Category: " << top.category
        << " | Description: " << top.description
        << " | Priority: " << top.priority << endl;
}
}

void processComplaint() {
if (complaintHeap.isEmpty()) {
    cout << "No complaints to process.\n";
    return;
}

Complaint top = complaintHeap.extractMin();
cout << "Processing Complaint ID: " << top.id
    << " | Category: " << top.category
    << " | Description: " << top.description
    << " | Priority: " << top.priority << endl;
}

////////////////////////////////////
// RESOURCE MODULE

void checkResources() {
cout << "\nAvailable Trucks:\n";
cout << left << setw(15) << "Truck ID" << "Capacity (kg)\n";
cout << string(25, '-') << endl;
displayTrucks(truckRoot);
}

void planTruckTripsWithAssumedTrucks(Graph& graph, int additionalTrucks, int capacity) {
cout << "\nPlanning truck trips with assumed additional trucks:\n";

TruckNode* originalTruckRoot = truckRoot;

vector<int> originalGarbage(graph.areaCount);
for (int i = 0; i < graph.areaCount; ++i) {
    originalGarbage[i] = graph.areas[i].garbage;
}
truckRoot = nullptr;

for (int i = 0; i < additionalTrucks; ++i) {
    string truckID = "T" + to_string(i + 1);
    truckRoot = insertTruck(truckRoot, truckID, capacity);
}

int totalAreas = graph.areaCount - 1;
int totalTrips = 0;
vector<int> truckTrips(additionalTrucks, 0);
vector<int> truckDistances(additionalTrucks, 0);
int totalDistanceAllAssumedTrucks = 0;

bool garbageLeft = true;

```

```

while (garbageLeft) {
    garbageLeft = false;
    bool allGarbageCollected = true;
    int totalGarbageCollected = 0;

    displayTargetedAreas(graph);
    cout << "\n\n";
    cout << "\tAvailable Trucks" << endl;
    cout << "-----\n\n";
    displayTrucks(truckRoot);

    for (int truckIndex = 0; truckIndex < additionalTrucks; ++truckIndex) {
        string truckID = getTruckID(truckRoot, truckIndex);
        int truckCapacity = getTruckCapacity(truckRoot, truckIndex);
        int remainingCapacity = truckCapacity;

        vector<pair<string, int>> route;
        route.push_back({ "Dumping Station", 0 });

        int currentAreaIndex = 0;
        int totalDistance = 0;

        while (remainingCapacity > 0) {
            int nextAreaIndex = -1;
            int minDistance = INF;
            int maxGarbage = 0;

            for (int areaIndex = 1; areaIndex < graph.areaCount; ++areaIndex) {
                if (graph.areas[areaIndex].garbage > 0) {
                    vector<int> path = dijkstra(graph, currentAreaIndex, areaIndex);
                    int distance = path.size() - 1;
                    if (distance < minDistance || (distance == minDistance &&
graph.areas[areaIndex].garbage > maxGarbage)) {
                        minDistance = distance;
                        nextAreaIndex = areaIndex;
                        maxGarbage = graph.areas[areaIndex].garbage;
                    }
                }
            }

            if (nextAreaIndex == -1) {
                break;
            }
            int garbage = graph.areas[nextAreaIndex].garbage;
            int load = min(garbage, remainingCapacity);
            remainingCapacity -= load;
            graph.areas[nextAreaIndex].garbage -= load;
            totalGarbageCollected += load;

            vector<int> pathToNextArea = dijkstra(graph, currentAreaIndex, nextAreaIndex);
            for (int areaIndex : pathToNextArea) {
                if (areaIndex != currentAreaIndex) {
                    route.push_back({ graph.areas[areaIndex].name, 0 });
                    totalDistance += graph.areas[currentAreaIndex].head->distance;
                    currentAreaIndex = areaIndex;
                }
            }

            route.back().second = load;

            if (graph.areas[nextAreaIndex].garbage > 0) {
                garbageLeft = true;
            }
        }
        vector<int> pathToDumpingStation = dijkstra(graph, currentAreaIndex, 0);
        for (int areaIndex : pathToDumpingStation) {
            if (areaIndex != currentAreaIndex) {

```

```

        route.push_back({ graph.areas[areaIndex].name, 0 });
        totalDistance += graph.areas[currentAreaIndex].head->distance;
        currentAreaIndex = areaIndex;
    }
}
if (route.size() > 1 && route.back().first == "Dumping Station") {
    route.pop_back();
}
if (route.size() > 1) {
    truckTrips[truckIndex]++;
    totalTrips++;
    truckDistances[truckIndex] += totalDistance;
    totalDistanceAllAssumedTrucks += totalDistance;
    cout << "\nTruck ID: " << truckID << "\nRoute: \n";
    for (const auto& {area, collected} : route) {
        cout << area << " (" << collected << " kg) -> ";
    }
    cout << "Dumping Station (0 kg)\n\n";
    cout << "Total Distance Traveled: " << totalDistance << " km\n";
    cout << "Truck Capacity: " << truckCapacity << " kg, Returning with: " << remainingCapacity << " kg
Capacity Left\n\n";
}
}
for (int areaIndex = 1; areaIndex < graph.areaCount; ++areaIndex) {
    if (graph.areas[areaIndex].garbage > 0) {
        allGarbageCollected = false;
        break;
    }
}

if (allGarbageCollected || totalGarbageCollected == 0) {
    break;
}
}
cout << "All areas serviced and garbage delivered to the dumping station.\n\n";

vector<pair<int, pair<int, int>>> truckData;
for (int truckIndex = 0; truckIndex < additionalTrucks; ++truckIndex) {
    string truckID = getTruckID(truckRoot, truckIndex);
    int truckIDNum = stoi(truckID.substr(1));
    truckData.push_back({ truckIDNum, { truckTrips[truckIndex], truckDistances[truckIndex] } });
}

sort(truckData.begin(), truckData.end());

for (const auto& {truckIDNum, tripsAndDistance} : truckData) {
    cout << "Truck ID: T" << truckIDNum << " made " << tripsAndDistance.first << " trips and traveled " << tripsAndDistance.second
<< " km.\n";
}
cout << "\nTotal number of trips by all trucks: " << totalTrips << "\n";
cout << "Total distance traveled by all assumed trucks: " << totalDistanceAllAssumedTrucks << " km\n";

truckRoot = originalTruckRoot;

for (int i = 0; i < graph.areaCount; ++i) {
    graph.areas[i].garbage = originalGarbage[i];
}
}

void assumeMoreResources() {
    int additionalTrucks, capacity;
    cout << "Enter the number of additional trucks to assume: ";
    cin >> additionalTrucks;
    cout << "Enter the capacity of the truck: ";
    cin >> capacity;

    planTruckTripsWithAssumedTrucks(graph, additionalTrucks, capacity);
}

```

```

void recommendTrucks() {
    int totalGarbage = 0;
    for (int i = 1; i < graph.areaCount; ++i) {
        totalGarbage += graph.areas[i].garbage;
    }

    int truckCapacity = 2000;
    int recommendedTrucks = ceil(static_cast<double>(totalGarbage) / truckCapacity);

    cout << "\nRecommended number of trucks: " << recommendedTrucks << endl;
    cout << "It is recommended to have trucks with capacity of 2000 kg.\n";
}

void generateReport() {
    ofstream reportFile("report.txt", ios::out | ios::trunc);
    if (!reportFile) {
        cerr << "Error opening report file.\n";
        return;
    }
    cout << "Generating report...\n\n";
    cout << "\t\t\t\t\t REPORT\n";
    cout << "\t\t\t\t\t===== \n\n";

    double totalGarbage = 0;
    double totalFuelCost = 0;
    double fuelPricePerKm = 190;

    for (int i = 1; i < graph.areaCount; ++i) {
        totalGarbage += graph.areas[i].garbage;
    }

    totalFuelCost = totalDistanceAllTrucks * fuelPricePerKm;

    double averageDistancePerTruck = totalDistanceAllTrucks / getTruckCount(truckRoot);
    double dailyFuelCost = totalFuelCost;
    double monthlyFuelCost = dailyFuelCost * 30;

    double annualTrashInTons = (totalGarbage * 365) / 1000;

    ifstream complaintFile("complaints.txt");
    if (complaintFile) {
        string line;
        reportFile << "\nComplaints:\n";
        while (getline(complaintFile, line)) {
            reportFile << line << endl;
        }
        complaintFile.close();
    }

    reportFile.close();

    cout <<
    "=====
    =====" << endl;
    cout << "\n\nAnnual Trash Collected: " << totalGarbage * 365 << " kg (" << annualTrashInTons << " tons)\n";
    cout << "Monthly Trash Collected: " << totalGarbage * 30 << " kg\n";
    cout << "Daily Trash Collected: " << totalGarbage << " kg\n";
    cout << "\nTotal Fuel Cost: Rs. " << totalFuelCost << "\n";
    cout << "Total Distance Traveled in a Day: " << totalDistanceAllTrucks << " km\n";
    cout << "Total Distance Traveled in a Month: " << totalDistanceAllTrucks * 30 << " km\n";
    cout << "Average Distance Traveled per Truck: " << averageDistancePerTruck << " km\n";
    cout << "\n\nDaily Fuel Cost: Rs. " << dailyFuelCost << "\n";
    cout << "Monthly Fuel Cost: Rs. " << monthlyFuelCost << "\n";
    cout << "Total Trips: " << totalTrips << "\n";

```



```

cout <<
"=====
===== " << endl;

ifstream displayComplaintFile("complaints.txt");
if (displayComplaintFile) {
    string line;

    cout << "\n                      Complaints\n\n";
    while (getline(displayComplaintFile, line)) {
        cout << line << endl;
    }
    displayComplaintFile.close();
}

////////////////////////////////////
// main menu

void mainMenu() {
    int choice;
    do {
        cout << "\n\t\t\t SMART CITY MANGEMENT SYSTEM";
        cout << "\n\t\t\t===== \n\n";

        cout << "1. Manage Areas" << endl;
        cout << "2. Manage Trucks" << endl;
        cout << "3. Add Connections Between Areas" << endl;
        cout << "4. Display Areas Connection" << endl;
        cout << "5. Explore City Map" << endl;
        cout << "6. Plan Truck Trips" << endl;
        cout << "7. Complain Mangement\n";
        cout << "8. Resource Mangement\n";
        cout << "9. Generate report\n";
        cout << "10. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                Areamenu();
                break;
            case 2:
                truckMenu();
                break;
            case 3:
            {
                totalGarbage = 0;
                cout << "\n\n";
                cout << left << setw(20) << "Area Name"
                    << setw(15) << "Population"
                    << setw(20) << "Waste Growth Rate"
                    << "Garbage per Day" << endl;
                cout << string(75, '-') << endl;
                displayAreas(root, totalGarbage);

                cout << "\nTotal City Garbage per Day: " << fixed << setprecision(2) << totalGarbage << " kg (Tons: " << totalGarbage /
1000 << " )\n\n";
                addAreasFromAVL(graph, root);
                string source, destination;
                int distance;
                cout << "Enter source area name: ";
                cin >> source;
                cout << "Enter destination area name: ";
                cin >> destination;
                cout << "Enter distance between areas (in km): ";
                cin >> distance;
                addConnection(graph, source, destination, distance);
                break;
            }
        }
    } while (choice != 10);
}

```



```

                exit(0);
                break;
            case 10:
                cout << "Exiting system...\n";
                exit(0);
                break;
            default:
                cout << "Invalid choice. Please try again.\n";
        }
    } while (choice != 10);
}

int main() {
    root = insertArea(root, "NAZIMABAD", 1000, 0.5);
    root = insertArea(root, "5-STAR", 2000, 0.6);
    root = insertArea(root, "NAGAN", 2000, 0.6);
    root = insertArea(root, "LIAQATABAD", 1500, 1.5);
    root = insertArea(root, "HIGHWAY", 2000, 0.6);
    root = insertArea(root, "GULSHAN", 2000, 0.8);
    root = insertArea(root, "SAFOORA", 3000, 0.7);
    root = insertArea(root, "MALIR", 2500, 0.9);
    root = insertArea(root, "HADEED", 1500, 0.6);
    root = insertArea(root, "KARSAZ", 2000, 0.8);
    root = insertArea(root, "SADDAR", 1000, 0.5);
    root = insertArea(root, "AIRPORT", 2000, 0.6);
    root = insertArea(root, "MAZAR-E-QUAID", 1500, 1.5);

    truckRoot = insertTruck(truckRoot, "T1", 1000);
    truckRoot = insertTruck(truckRoot, "T2", 2000);
    truckRoot = insertTruck(truckRoot, "T3", 4500);
    truckRoot = insertTruck(truckRoot, "T4", 1000);
    initializeGraph(graph);
    addAreasFromAVL(graph, root);

    addConnection(graph, "Dumping Station", "NAZIMABAD", 6);
    addConnection(graph, "5-STAR", "NAGAN", 5);
    addConnection(graph, "5-STAR", "NAZIMABAD", 2);
    addConnection(graph, "NAGAN", "HIGHWAY", 4);
    addConnection(graph, "HIGHWAY", "LIAQATABAD", 7);
    addConnection(graph, "HIGHWAY", "HADEED", 13);
    addConnection(graph, "LIAQATABAD", "5-STAR", 4);
    addConnection(graph, "LIAQATABAD", "NAZIMABAD", 5);
    addConnection(graph, "LIAQATABAD", "MAZAR-E-QUAID", 8);
    addConnection(graph, "LIAQATABAD", "GULSHAN", 7);
    addConnection(graph, "KARSAZ", "GULSHAN", 2);
    addConnection(graph, "GULSHAN", "SAFOORA", 8);
    addConnection(graph, "SAFOORA", "MALIR", 6);
    addConnection(graph, "HADEED", "MALIR", 3);
    addConnection(graph, "MALIR", "KARSAZ", 9);
    addConnection(graph, "KARSAZ", "SADDAR", 10);
    addConnection(graph, "SADDAR", "MAZAR-E-QUAID", 3);
    addConnection(graph, "GULSHAN", "MAZAR-E-QUAID", 6);
    addConnection(graph, "AIRPORT", "KARSAZ", 6);
    addConnection(graph, "AIRPORT", "MALIR", 3);
    system("cls");
    mainMenu();
    return 0;
}

```

12) Output:

```
Microsoft Visual Studio Debug Console

SMART CITY MANGEMENT SYSTEM
=====

1. Manage Areas
2. Manage Trucks
3. Add Connections Between Areas
4. Display Areas Connection
5. Explore City Map
6. Plan Truck Trips
7. Complain Mangement
8. Resource Management
9. Generate report
10. Exit
Enter your choice: 1

Area Mangement Potal
=====

1. Add Area
2. Display Areas
3. Display Total City Waste
4. Main menu
5. Exit
Enter your choice: 2

Area Name      Population      Waste Growth Rate      Garbage per Day      kg
-----
5-STAR         2000           0.60                   1200.00              kg
AIRPORT        2000           0.60                   1200.00              kg
GULSHAN        2000           0.80                   1600.00              kg
HADEED         1500           0.60                   900.00               kg
HIGHWAY        2000           0.60                   1200.00              kg
KARSAZ         2000           0.80                   1600.00              kg
LIAQATABAD     1500           1.50                   2250.00              kg
MALIR          2500           0.90                   2250.00              kg
MAZAR-E-QUAID  1500           1.50                   2250.00              kg
NAGAN          2000           0.60                   1200.00              kg
NAZIMABAD      1000           0.50                   500.00               kg
SADDAR         1000           0.50                   500.00               kg
SAFOORA        3000           0.70                   2100.00              kg
```

```
Microsoft Visual Studio Debug Console

Area Mangement Potal
=====

1. Add Area
2. Display Areas
3. Display Total City Waste
4. Main menu
5. Exit
Enter your choice: 1
Enter area name: BALDIA
Enter population: 1800
Enter waste growth rate (kg per person per day): 0.5
Area added successfully!
```

```

Microsoft Visual Studio Debug Console

Area Mangement Potal
=====

1. Add Area
2. Display Areas
3. Display Total City Waste
4. Main menu
5. Exit
Enter your choice: 2

Area Name      Population      Waste Growth Rate      Garbage per Day
-----
5-STAR         2000           0.60                   1200.00             kg
AIRPORT        2000           0.60                   1200.00             kg
BALDIA         1800           0.50                   900.00              kg
GULSHAN        2000           0.80                   1600.00             kg
HADEED         1500           0.60                   900.00              kg
HIGHWAY        2000           0.60                   1200.00             kg
KARSAZ         2000           0.80                   1600.00             kg
LIAQATABAD     1500           1.50                   2250.00             kg
MALIR          2500           0.90                   2250.00             kg
MAZAR-E-QUAID  1500           1.50                   2250.00             kg
NAGAN          2000           0.60                   1200.00             kg
NAZIMABAD      1000           0.50                   500.00              kg
SADDAR         1000           0.50                   500.00              kg
SAFOORA        3000           0.70                   2100.00             kg

```

```

Microsoft Visual Studio Debug Console

Area Mangement Potal
=====

1. Add Area
2. Display Areas
3. Display Total City Waste
4. Main menu
5. Exit
Enter your choice: 3

Area Name      Population      Waste Growth Rate      Garbage per Day
-----
5-STAR         2000           0.60                   1200.00             kg
AIRPORT        2000           0.60                   1200.00             kg
BALDIA         1800           0.50                   900.00              kg
GULSHAN        2000           0.80                   1600.00             kg
HADEED         1500           0.60                   900.00              kg
HIGHWAY        2000           0.60                   1200.00             kg
KARSAZ         2000           0.80                   1600.00             kg
LIAQATABAD     1500           1.50                   2250.00             kg
MALIR          2500           0.90                   2250.00             kg
MAZAR-E-QUAID  1500           1.50                   2250.00             kg
NAGAN          2000           0.60                   1200.00             kg
NAZIMABAD      1000           0.50                   500.00              kg
SADDAR         1000           0.50                   500.00              kg
SAFOORA        3000           0.70                   2100.00             kg

Total City Garbage per Day: 19650.00 kg (Tons: 19.65 )

```

```
SMART CITY MANGEMENT SYSTEM
=====

1. Manage Areas
2. Manage Trucks
3. Add Connections Between Areas
4. Display Areas Connection
5. Explore City Map
6. Plan Truck Trips
7. Complain Mangement
8. Resource Management
9. Generate report
10. Exit
Enter your choice: 2

Trucks Management System
=====

1. Add Truck
2. Display Trucks
3. Exit to Main Menu
Enter your choice: 2
Truck ID      Capacity
-----
T1             1000      kg
T2             2000      kg
T3             4500      kg
T4             1000      kg
```

```
Microsoft Visual Studio Debug Console

Trucks Management System
=====

1. Add Truck
2. Display Trucks
3. Exit to Main Menu
Enter your choice: 1
Enter truck ID: T5
Enter truck capacity (in kg): 1200

--- Inserting truck: T5 with capacity 1200 kg. ---
Truck added successfully!
```

```

Microsoft Visual Studio Debug Console

SMART CITY MANGEMENT SYSTEM
=====

1. Manage Areas
2. Manage Trucks
3. Add Connections Between Areas
4. Display Areas Connection
5. Explore City Map
6. Plan Truck Trips
7. Complain Mangement
8. Resource Management
9. Generate report
10. Exit
Enter your choice: 3

Area Name      Population    Waste Growth Rate    Garbage per Day
-----
5-STAR         2000         0.60                 1200.00          kg
AIRPORT        2000         0.60                 1200.00          kg
BALDIA         1800         0.50                 900.00           kg
GULSHAN        2000         0.80                 1600.00          kg
HADEED         1500         0.60                 900.00           kg
HIGHWAY        2000         0.60                 1200.00          kg
KARSAZ         2000         0.80                 1600.00          kg
LIAQATABAD     1500         1.50                 2250.00          kg
MALIR          2500         0.90                 2250.00          kg
MAZAR-E-QUAID  1500         1.50                 2250.00          kg
NAGAN          2000         0.60                 1200.00          kg
NAZIMABAD      1000         0.50                 500.00           kg
SADDAR         1000         0.50                 500.00           kg
SAFOORA        3000         0.70                 2100.00          kg

Total City Garbage per Day: 19650.00 kg (Tons: 19.65 )

Enter source area name: BALDIA
Enter destination area name: NAZIMABAD
Enter distance between areas (in km): 6
Connection added between BALDIA and NAZIMABAD with distance 6 km.

```

```

Microsoft Visual Studio Debug Console

SMART CITY MANGEMENT SYSTEM
=====

1. Manage Areas
2. Manage Trucks
3. Add Connections Between Areas
4. Display Areas Connection
5. Explore City Map
6. Plan Truck Trips
7. Complain Mangement
8. Resource Management
9. Generate report
10. Exit
Enter your choice: 4

Areas and their connections:
Dumping Station (Garbage: 0 kg) -> NAZIMABAD(6 km)
5-STAR (Garbage: 1200 kg) -> LIAQATABAD(4 km) NAZIMABAD(2 km) NAGAN(5 km)
AIRPORT (Garbage: 1200 kg) -> MALIR(3 km) KARSAZ(6 km)
GULSHAN (Garbage: 1600 kg) -> MAZAR-E-QUAID(6 km) SAFOORA(8 km) KARSAZ(2 km) LIAQATABAD(7 km)
HADEED (Garbage: 900 kg) -> MALIR(3 km) HIGHWAY(13 km)
HIGHWAY (Garbage: 1200 kg) -> HADEED(13 km) LIAQATABAD(7 km) NAGAN(4 km)
KARSAZ (Garbage: 1600 kg) -> AIRPORT(6 km) SADDAR(10 km) MALIR(9 km) GULSHAN(2 km)
LIAQATABAD (Garbage: 2250 kg) -> GULSHAN(7 km) MAZAR-E-QUAID(8 km) NAZIMABAD(5 km) 5-STAR(4 km) HIGHWAY(7 km)
MALIR (Garbage: 2250 kg) -> AIRPORT(3 km) KARSAZ(9 km) HADEED(3 km) SAFOORA(6 km)
MAZAR-E-QUAID (Garbage: 2250 kg) -> GULSHAN(6 km) SADDAR(3 km) LIAQATABAD(8 km)
NAGAN (Garbage: 1200 kg) -> HIGHWAY(4 km) 5-STAR(5 km)
NAZIMABAD (Garbage: 500 kg) -> BALDIA(6 km) LIAQATABAD(5 km) 5-STAR(2 km) Dumping Station(6 km)
SADDAR (Garbage: 500 kg) -> MAZAR-E-QUAID(3 km) KARSAZ(10 km)
SAFOORA (Garbage: 2100 kg) -> MALIR(6 km) GULSHAN(8 km)
BALDIA (Garbage: 900 kg) -> NAZIMABAD(6 km)

```


Truck ID: T1
Route:
Dumping Station (0 kg) -> NAZIMABAD (500 kg) -> LIAQATABAD (500 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 25 km
Truck Capacity: 1000 kg, Returning with: 0 kg Capacity Left

Truck ID: T2
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (1750 kg) -> MAZAR-E-QUAID (250 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 38 km
Truck Capacity: 2000 kg, Returning with: 0 kg Capacity Left

Truck ID: T3
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> 5-STAR (1200 kg) -> NAGAN (1200 kg) -> HIGHWAY (1200 kg) -> HADEED (900 kg) -> HIGHWAY (0 kg) -> NAGAN (0 kg) -> 5-STAR (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 63 km
Truck Capacity: 4500 kg, Returning with: 0 kg Capacity Left

Truck ID: T4
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> BALDIA (900 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> MAZAR-E-QUAID (100 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 50 km
Truck Capacity: 1000 kg, Returning with: 0 kg Capacity Left

Truck ID: T5
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> MAZAR-E-QUAID (1200 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 38 km
Truck Capacity: 1200 kg, Returning with: 0 kg Capacity Left

Targeted Areas for Collecting Trash:

Area Name	Garbage (kg)
AIRPORT	1200
GULSHAN	1600
KARSAZ	1600
MALIR	2250
MAZAR-E-QUAID	700
SADDAR	500
SAFOORA	2100

Available Trucks

T1	1000	kg
T2	2000	kg
T3	4500	kg
T4	1000	kg
T5	1200	kg

Truck ID: T1
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> GULSHAN (1000 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 38 km
Truck Capacity: 1000 kg, Returning with: 0 kg Capacity Left

Truck ID: T2
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> MAZAR-E-QUAID (700 kg) -> GULSHAN (600 kg) -> SAFOORA (700 kg) -> GULSHAN (0 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 56 km
Truck Capacity: 2000 kg, Returning with: 0 kg Capacity Left

Truck ID: T3
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> GULSHAN (0 kg) -> KARSAZ (1600 kg) -> MALIR (2250 kg) -> SAFOORA (650 kg) -> GULSHAN (0 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 59 km
Truck Capacity: 4500 kg, Returning with: 0 kg Capacity Left

Truck ID: T4
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> GULSHAN (0 kg) -> SAFOORA (750 kg) -> MALIR (0 kg) -> AIRPORT (250 kg) -> KARSAZ (0 kg) -> GULSHAN (0 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 62 km
Truck Capacity: 1000 kg, Returning with: 0 kg Capacity Left

Truck ID: T5
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> MAZAR-E-QUAID (0 kg) -> SADDAR (500 kg) -> KARSAZ (0 kg) -> AIRPORT (700 kg) -> KARSAZ (0 kg) -> GULSHAN (0 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)
Total Distance Traveled: 62 km
Truck Capacity: 1200 kg, Returning with: 0 kg Capacity Left

Targeted Areas for Collecting Trash:

Area Name	Garbage (kg)
AIRPORT	250

```

Available Trucks
-----
T1          1000    kg
T2          2000    kg
T3          4500    kg
T4          1000    kg
T5          1200    kg

Truck ID: T1
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> GULSHAN (0 kg) -> KARSAZ (0 kg) -> AIRPORT (250 kg) -> KARSAZ (0 kg) -> GULSHAN (0 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 59 km
Truck Capacity: 1000 kg, Returning with: 750 kg Capacity Left

All areas serviced and garbage delivered to the dumping station.

Truck ID: T1 made 3 trips and traveled 122 km.
Truck ID: T2 made 2 trips and traveled 94 km.
Truck ID: T3 made 2 trips and traveled 122 km.
Truck ID: T4 made 2 trips and traveled 112 km.
Truck ID: T5 made 2 trips and traveled 100 km.

Total number of trips by all trucks: 11
Total distance traveled by all trucks: 550 km

```

```

Enter your choice: 7

Complaint Management System
=====

1. Register Complaint
2. Display Complaints
3. Process Complaint
4. Main Menu
5. Exit
Enter your choice: 2
No complaints registered yet.

Complaint Management System
=====

1. Register Complaint
2. Display Complaints
3. Process Complaint
4. Main Menu
5. Exit
Enter your choice: 1

Select category:
1. Vehicle
2. Service
3. Waste Collection
4. Suggestion
Enter your choice: 1
Enter description: T1 IS OVERFILLED
Enter urgency level (1-5): 2
Complaint registered successfully!

```

```

Complaint Management System
=====

1. Register Complaint
2. Display Complaints
3. Process Complaint
4. Main Menu
5. Exit
Enter your choice: 1

Select category:
1. Vehicle
2. Service
3. Waste Collection
4. Suggestion
Enter your choice: 3
Enter description: WASTE IS NOT COLLECTED FROM KARSAZ
Enter urgency level (1-5): 1
Complaint registered successfully!

```

```

Complaint Management System
=====

1. Register Complaint
2. Display Complaints
3. Process Complaint
4. Main Menu
5. Exit
Enter your choice: 2

--- Complaints (Prioritized by Urgency) ---
ID: 2 | Category: Waste Collection | Description: WASTE IS NOT COLLECTED FROM KARSAZ | Priority: 1
ID: 1 | Category: Vehicle | Description: T1 IS OVERFILLED | Priority: 2

Complaint Management System
=====

1. Register Complaint
2. Display Complaints
3. Process Complaint
4. Main Menu
5. Exit
Enter your choice: 3
Processing Complaint ID: 2 | Category: Waste Collection | Description: WASTE IS NOT COLLECTED FROM KARSAZ | Priority: 1

```

```

Complaint Management System
=====

1. Register Complaint
2. Display Complaints
3. Process Complaint
4. Main Menu
5. Exit
Enter your choice: 3
Processing Complaint ID: 1 | Category: Vehicle | Description: T1 IS OVERFILLED | Priority: 2

Complaint Management System
=====

1. Register Complaint
2. Display Complaints
3. Process Complaint
4. Main Menu
5. Exit
Enter your choice: 2
No complaints registered yet.

```

```

Enter your choice: 8

Resource Management Portal
=====

1. Check resources
2. Assume more resources
3. Recommend number of trucks
4. Exit to Main Menu
Enter your choice: 1

Available Trucks:
Truck ID      Capacity (kg)
-----
T1             1000      kg
T2             2000      kg
T3             4500      kg
T4             1000      kg
T5             1200      kg

```

```

Enter your choice: 2
Enter the number of additional trucks to assume: 2
Enter the capacity of the truck: 500

Planning truck trips with assumed additional trucks:

--- Inserting truck: T1 with capacity 500 kg. ---
--- Inserting truck: T2 with capacity 500 kg. ---

Targeted Areas for Collecting Trash:
Area Name          Garbage (kg)
-----
5-STAR             1200
AIRPORT            1200
GULSHAN            1600
HADEED             900
HIGHWAY            1200
KARSAZ             1600
LIAQATABAD         2250
MALIR              2250
MAZAR-E-QUAID      2250
NAGAN              1200
NAZIMABAD          500
SADDAR             500
SAFOORA            2100
BALDIA             900

```

```

Available Trucks
-----
T1          500      kg
T2          500      kg

Truck ID: T1
Route:
Dumping Station (0 kg) -> NAZIMABAD (500 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 12 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

Truck ID: T2
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (500 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 25 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

```

```

Targeted Areas for Collecting Trash:
Area Name          Garbage (kg)
-----
5-STAR             1200
AIRPORT            1200
GULSHAN            1600
HADEED             900
HIGHWAY            1200
KARSAZ             1600
LIAQATABAD         1750
MALIR              2250
MAZAR-E-QUAID      2250
NAGAN              1200
SADDAR             500
SAFOORA            2100
BALDIA             900

Available Trucks
-----
T1          500      kg
T2          500      kg

Truck ID: T1
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (500 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 25 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

Truck ID: T2
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (500 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 25 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

```

```

Targeted Areas for Collecting Trash:
Area Name          Garbage (kg)
-----
5-STAR             1200
AIRPORT            1200
GULSHAN            1600
HADEED             900
HIGHWAY            1200
KARSAZ             1600
LIAQATABAD         750
MALIR              2250
MAZAR-E-QUAID      2250
NAGAN              1200
SADDAR             500
SAFOORA            2100
BALDIA             900

Available Trucks
-----
T1          500      kg
T2          500      kg

Truck ID: T1
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> 5-STAR (500 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 22 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

Truck ID: T2
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> BALDIA (500 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 24 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

```

Targeted Areas for Collecting Trash:

Area Name	Garbage (kg)
5-STAR	700
AIRPORT	1200
GULSHAN	1600
HADEED	900
HIGHWAY	1200
KARSAZ	1600
LIAQATABAD	750
MALIR	2250
MAZAR-E-QUAID	2250
NAGAN	1200
SADDAR	500
SAFOORA	2100
BALDIA	400

Available Trucks

T1	500	kg
T2	500	kg

Truck ID: T1

Route:

Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (500 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 25 km

Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

Truck ID: T2

Route:

Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> 5-STAR (500 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 22 km

Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

Targeted Areas for Collecting Trash:

Area Name	Garbage (kg)
5-STAR	200
AIRPORT	1200
GULSHAN	1600
HADEED	900
HIGHWAY	1200
KARSAZ	1600
LIAQATABAD	250
MALIR	2250
MAZAR-E-QUAID	2250
NAGAN	1200
SADDAR	500
SAFOORA	2100
BALDIA	400

Available Trucks

T1	500	kg
T2	500	kg

Truck ID: T1

Route:

Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> BALDIA (400 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (100 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 37 km

Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

Truck ID: T2

Route:

Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> 5-STAR (200 kg) -> NAGAN (300 kg) -> 5-STAR (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 30 km

Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left

```

Targeted Areas for Collecting Trash:
Area Name           Garbage (kg)
-----
AIRPORT             1200
GULSHAN             1600
HADEED              900
HIGHWAY             1200
KARSAZ              1600
LIAQATABAD          150
MALIR               2250
MAZAR-E-QUAID       2250
NAGAN               900
SADDAR              500
SAFOORA             2100


Available Trucks
-----
T1          500      kg
T2          500      kg

Truck ID: T1
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (150 kg) -> MAZAR-E-QUAID (350 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 38 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left


Truck ID: T2
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> MAZAR-E-QUAID (500 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 38 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left


Available Trucks
-----
T1          500      kg
T2          500      kg

Truck ID: T1
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> GULSHAN (0 kg) -> KARSAZ (0 kg) -> MALIR (450 kg) -> AIRPORT (50 kg) -> KARSAZ (0 kg) -> GULSHAN (0 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 62 km
Truck Capacity: 500 kg, Returning with: 0 kg Capacity Left


Truck ID: T2
Route:
Dumping Station (0 kg) -> NAZIMABAD (0 kg) -> LIAQATABAD (0 kg) -> GULSHAN (0 kg) -> KARSAZ (0 kg) -> AIRPORT (150 kg) -> KARSAZ (0 kg) -> GULSHAN (0 kg) -> LIAQATABAD (0 kg) -> NAZIMABAD (0 kg) -> Dumping Station (0 kg)

Total Distance Traveled: 59 km
Truck Capacity: 500 kg, Returning with: 350 kg Capacity Left


All areas serviced and garbage delivered to the dumping station.


Truck ID: T1 made 20 trips and traveled 857 km.
Truck ID: T2 made 20 trips and traveled 878 km.


Total number of trips by all trucks: 40
Total distance traveled by all assumed trucks: 1735 km

```

```
3. Recommend number of trucks
4. Exit to Main Menu
Enter your choice: 3

Recommended number of trucks: 10
It is recommended to have trucks with capacity of 2000 kg.
```

```
Resource Management Portal
=====
```

```
1. Check resources
2. Assume more resources
3. Recommend number of trucks
4. Exit to Main Menu
Enter your choice: 4
Exiting to main menu...
```

```
SMART CITY MANGEMENT SYSTEM
=====
```

```
1. Manage Areas
2. Manage Trucks
3. Add Connections Between Areas
4. Display Areas Connection
5. Explore City Map
6. Plan Truck Trips
7. Complain Mangement
8. Resource Management
9. Generate report
10. Exit
Enter your choice: 9
Generating report...
```

```
Enter your choice: 9
Generating report...
```

```
REPORT
=====
```

```
=====
Annual Trash Collected: 7172250.00 kg (7172.25 tons)
Monthly Trash Collected: 589500.00 kg
Daily Trash Collected: 19650.00 kg
```

```
Total Fuel Cost: Rs. 104500.00
Total Distance Traveled in a Day: 550 km
Total Distance Traveled in a Month: 16500 km
Average Distance Traveled per Truck: 110.00 km
```

```
Daily Fuel Cost: Rs. 104500.00
Monthly Fuel Cost: Rs. 3135000.00
Total Trips: 11
```

```
=====
Complaints
```

```
ID: 1 | Category: Vehicle | Description: T1 IS OVERFILLED | Priority: 2
ID: 2 | Category: Waste Collection | Description: WASTE IS NOT COLLECTED FROM KARSAB | Priority: 1
```

```
=====
Report generated successfully
```


13) Conclusion:

- **Reflect on Objectives and Lessons Learned**

The project met its objectives by creating an efficient, scalable, and modular waste management system. It demonstrated the effectiveness of data structures and algorithms in solving real-world problems. Key lessons included the importance of modular design, handling edge cases, and optimizing computational efficiency.

- **Future Improvements:**

1. Integrating real-time GPS tracking for dynamic route updates.
2. Implementing IoT-based sensors to monitor garbage levels in real-time.
3. Enhancing the user interface for better accessibility.
4. Expanding reporting features to include real-time dashboards.

14) References:

Link Of Project File:

<https://drive.google.com/file/d/16jjorMTu854VCFwiaOdYtTNg5kh7K2xB/view?usp=drivesdk>